Data Structures and Algorithms

Lab 2 – Version 4

Sorting, Searching, and Sliding Windows

Fundamentals, Real-World Applications, and Interview Practice

Lab Overview

This lab introduces **sorting, linear search, binary search, and sliding windows** as techniques for organizing data, locating values, and processing consecutive elements efficiently. You will study how each technique works, trace its steps on small inputs, and apply it to practical programming problems.

You will implement bubble sort, selection sort, and insertion sort; compare linear and binary search; and use fixed-size and variable-size windows. Throughout the lab, identify the conditions that make each method appropriate, such as sorted input for binary search and the stated value restrictions for sum-based variable windows.

For each required solution, explain the main idea, justify why it works, analyze its time and auxiliary-space costs, and verify it using normal, boundary, and tricky test cases. This is a **one-week individual assignment**. Use **Java** for all implementations.

Assignment Scope

Component	What to complete
Core problems	Problems **1, 3, 4, 5, 7, 9, and 10**.
Applied choice	Choose **one** of Problems **2 or 8**.
Interview choice	Choose **one** of Problems **6 or 11**; submit **Java code and a report section**.
Written practice	Interview Corner A–D, the comparison table, technique-recognition exercise, and reflections: **PDF report only; no extra Java programs**.
Optional extension	Problem 12 and unselected choice problems.

Complete **nine required programming problems**: the seven core problems and one from each choice group. Some require multiple implementations. Optional work does not replace required work.

The three ideas

Sorting creates order. Binary search uses order to eliminate candidates. Sliding windows reuse information between overlapping consecutive regions. Use the property that fits the question; sorting is not a prerequisite for a window.

Programming and Analysis Requirements

Programming Requirements

- Implement the requested sorting and binary-search algorithms yourself. Built-in helpers such as `Arrays.sort()`, `Collections.sort()`, and library search methods may not replace the required implementations.
- Use arrays for array problems. `HashSet` is allowed for Problem 11; its optional map version may use `HashMap`. Do not replace requested linear or binary searches with hash lookups.
- Reuse your own helpers and include them in the source package. Each problem must have a small input driver or clearly labeled test driver; no menu is required.
- Pseudocode is a learning guide. Write the Java implementation and tests and explain your own decisions.

Conventions Used Throughout the Handout

- Indices are zero-based. A nonempty inclusive window $[L, \ldots, R]$ has length $R - L + 1$. Dates, row labels, and IDs are not automatically indices.
- Arrays and strings are non-null. Empty inputs are allowed where stated. An unsuccessful exact-index search returns -1; a lower-bound position can be n.
- Use `int` for data and indices; use `long` for sums, targets, budgets, pair counts, and operation counters. Unless restricted further, use at most 10^6 values with $|a[i]| \leq 10^9$. Limit quadratic sorts to the small sizes specified in their problems.
- Fixed windows require $1 \leq k \leq n$. Reject invalid k with a clear message or `IllegalArgumentException`. Other inputs are assumed to satisfy stated contracts; parsing malformed text is not the learning objective.
- Preserve original arrays unless mutation is requested. When an answer includes a tied best window, return the earliest start. Never sort a sequence whose original adjacency is part of the question.

Analysis and Testing

For each completed problem, give its idea, a short correctness argument, time and auxiliary-space costs, answers to the questions, and at least three tests: **normal, boundary, and tricky**. Cover the specifically named cases too.

Record input, expected output, and actual output. Compare required approaches on the same original values. Identify output storage separately from working memory. Include preprocessing in the complete cost. Timing is optional; operation counters are required only where specified.

Review 1 – Order, Contiguity, and Complexity

Sorting and Searching

Sorting rearranges values by a key, such as weight, timestamp, or ID. Searching asks whether a target occurs or where a relevant position lies. Creating order has a cost even when it makes later queries easier.

Move a complete record whenever its key moves. Sorting registration IDs independently from their associated booking details would corrupt those relationships.

Subarray, Substring, and Subsequence

A **subarray** uses consecutive array positions. A **substring** uses consecutive string positions. A **subsequence** may skip positions while keeping the selected positions in their original order. For $[4, 2, 7, 1]$, $[2, 7]$ is a subarray; $[4, 7]$ is a subsequence but not a subarray.

Complexity Refresher

Growth	Interpretation and example
$O(1)$	Constant work or working storage; access one array value.
$O(\log n)$	Repeatedly halve a candidate interval; binary search.
$O(n)$	Inspect every value a constant number of times.
$O(n \log n)$	A common cost of efficient general-purpose sorting.
$O(n^2)$	A quadratic number of comparisons in elementary sorts.
$O(nk)$	An upper bound for independently summing length-k windows.

Growth, not stopwatch time

Best, average, and worst cases can differ. A nested loop is not automatically quadratic: if both boundaries advance at most n times in total, their total movement is linear. A few variables use constant working space; a protective array copy uses space proportional to its length.

Useful Correctness Questions

Which region is already sorted? Where could a target still occur? Which values are in the current window? What does the running sum represent? A statement that remains true as the algorithm proceeds is called an **invariant**.

Connection to Lab 1

Lab 1 reused suffix maxima and counted contributions across subarrays. Here sorted order supports searching and fixed windows reuse neighboring totals. A suffix maximum cannot in general be subtracted from another suffix maximum to recover an interval maximum. The stored summary must match the question.

Review 2 – Bubble Sort

Idea and When to Apply It

Compare neighboring values and swap an inverted pair. A left-to-right pass moves the largest remaining value to the end of the unsorted region. Repeat with that region shortened. Use this algorithm to study adjacent exchanges, loop boundaries, and early termination on very small inputs.

Worked Trace

Starting array: $[6, 2, 5, 1, 7]$.

First-pass comparison	Array after it
6 and 2: swap	$[2, 6, 5, 1, 7]$
6 and 5: swap	$[2, 5, 6, 1, 7]$
6 and 1: swap	$[2, 5, 1, 6, 7]$
6 and 7: keep	$[2, 5, 1, 6, 7]$

The suffix ending in 7 is fixed. Subsequent passes produce $[2, 1, 5, 6, 7]$, then $[1, 2, 5, 6, 7]$. A following no-swap pass confirms completion.

Pseudocode

```
for end from n - 1 down to 1:
    swapped = false
    for j from 0 to end - 1:
        if a[j] > a[j + 1]:
            swap a[j] and a[j + 1]
            swapped = true
    if swapped is false:
        stop
```

Correctness and Cost

Every complete pass fixes one largest remaining value, extending the sorted suffix. A no-swap pass means that all adjacent pairs in the remaining region are already ordered. Reset the flag for every pass.

Optimized bubble sort uses $O(n)$ best-case time on sorted input and $O(n^2)$ average/worst-case time. Working space is $O(1)$. The strict comparison `>` preserves the order of equal-key records: this is **stability**. Empty and one-element inputs require no comparisons.

Rearrangement and sorting

Interleaving two halves preserves each half's relative order but alternates their positions. Sorting uses key comparisons instead. Do not treat every positional rearrangement as a sort.

Review 3 – Selection Sort

Idea and When to Apply It

Find the minimum of the unsorted suffix and swap it into the next output position. This is easy to trace and limits swaps. For very small data, it is useful when reducing writes matters more than avoiding comparisons.

Worked Trace

Starting array: $[6, 2, 5, 1, 7]$.

Output index	Selected minimum	Array afterward
0	1 at index 3	$[1, 2, 5, 6, 7]$
1	2 at index 1	$[1, 2, 5, 6, 7]$
2	5 at index 2	$[1, 2, 5, 6, 7]$
3	6 at index 3	$[1, 2, 5, 6, 7]$

Skip a swap when the minimum is already in position. Do not stop scanning just because one smaller value has been found: it might not be the minimum.

Pseudocode

```
for i from 0 to n - 2:
    minIndex = i
    for j from i + 1 to n - 1:
        if a[j] < a[minIndex]:
            minIndex = j
    if minIndex != i:
        swap a[i] and a[minIndex]
```

Correctness and Cost

Before iteration i, the prefix before i contains the smallest values in sorted order. Selecting the next minimum extends that prefix by one. For n values the key comparisons total

$$(n-1) + (n-2) + \cdots + 1 = \frac{n(n-1)}{2}.$$

The usual implementation takes $O(n^2)$ time on all input orders, at most $n-1$ nontrivial swaps, and $O(1)$ auxiliary space.

Equal keys

The swap-based form is generally unstable. Starting with labeled records $[(3, A), (3, B), (1, C)]$, the first swap gives $[(1, C), (3, B), (3, A)]$. The two records with key 3 have changed relative order.

Review 4 – Insertion Sort

Idea and When to Apply It

Maintain a sorted prefix. Save the next value as `key`, shift larger prefix values right, and insert the key into the gap. This resembles arranging cards in a hand. It is useful for small or nearly sorted data.

Worked Trace

Starting array: $[6, 2, 5, 1, 7]$.

Key	Work	Array afterward
2	Shift 6; insert 2.	$[2, 6, 5, 1, 7]$
5	Shift 6; insert 5.	$[2, 5, 6, 1, 7]$
1	Shift 6, 5, and 2; insert 1.	$[1, 2, 5, 6, 7]$
7	No shift; place 7.	$[1, 2, 5, 6, 7]$

Pseudocode

```
for i from 1 to n - 1:
    key = a[i]
    j = i - 1
    while j >= 0 and a[j] > key:
        a[j + 1] = a[j]
        j = j - 1
    a[j + 1] = key
```

Correctness and Cost

The prefix is sorted before each insertion. Shifting only larger values preserves their relative order and opens the correct gap. Save the key before overwriting its original position, and check `j >= 0` before reading `a[j]`.

Best-case time is $O(n)$ on sorted input; average/worst-case time is $O(n^2)$. Reverse order produces many shifts. Working space is $O(1)$. Using `>` rather than `>=` preserves equal-key order.

Comparing the Three Sorts

Algorithm	Best	Worst	Aux. space
Optimized bubble	$O(n)$	$O(n^2)$	$O(1)$
Selection	$O(n^2)$	$O(n^2)$	$O(1)$
Insertion	$O(n)$	$O(n^2)$	$O(1)$

A protective copy is additional to these working-space bounds. Sorting does not become $O(n \log n)$ merely because it is used before a binary search.

Review 5 – Linear Search

Idea and When to Apply It

Inspect values in sequence until a match is found or the input ends. Linear search needs no sorted order. It is a reasonable first choice for a small collection, one query, or frequently changing data without an ordered index.

Worked Trace

Access codes: $[310, 145, 620, 275, 480]$. Target: 275.

Index	Value	Inspection	Action
0	310	1	Continue.
1	145	2	Continue.
2	620	3	Continue.
3	275	4	Return index 3.

Pseudocode for the First Occurrence

```
for i from 0 to n - 1:
    if a[i] == target:
        return i
return -1
```

Existence means the returned index is not -1. To find the last occurrence, continue and remember the latest match. To count occurrences, inspect all values and increment a counter for every match. Finding one match is not a complete frequency count.

Correctness and Cost

Before position i is inspected, all earlier positions have been checked. Returning a first match is therefore correct; completing the scan establishes absence. Best-case time for existence/first position is $O(1)$, and worst-case time is $O(n)$. A full counting scan is $O(n)$. Working space is $O(1)$.

Connect the scan to its question

Lab 1's first-unique-position problem needed complete frequencies before the earliest valid original position was known. A local threshold test is different: its first successful comparison can be enough.

For a single query, sorting first may be wasted work. For many queries on a fixed array, compare q scans with one preparation step and q lookups.

Review 6 – Binary Search

Idea and Required Condition

On an **ascending sorted random-access array**, compare the target with the midpoint. If the midpoint is too small, discard its left side and itself; if too large, discard its right side and itself. This supports repeated lookups on data whose sorted order is maintained.

Worked Trace

Array: $[3, 7, 11, 18, 24, 31, 42]$. Target: 24.

low	high	mid	a[mid]	Action
0	6	3	18	Set low to 4.
4	6	5	31	Set high to 4.
4	4	4	24	Return 4.

Pseudocode: Inclusive Boundaries

```
low = 0
high = n - 1
while low <= high:
    mid = low + (high - low) / 2   // integer division
    if a[mid] == target:
        return mid
    else if a[mid] < target:
        low = mid + 1
    else:
        high = mid - 1
return -1
```

Correctness and Cost

If a match exists, a matching position remains in $[low, \ldots, high]$. Sorted order justifies each removal. For example, if `a[mid] < target`, no value at or left of that midpoint can be the target. Moving past `mid` ensures that the candidate interval shrinks. An empty input starts with `high = -1`.

Best time is $O(1)$; worst time is $O(\log n)$; iterative working space is $O(1)$. A million positions require at most about 20 midpoint inspections. An inspection is one selected midpoint value, not each separate relational test.

An index belongs to its array

Searching a sorted copy returns a position in that copy, not the original registration order. Keep record IDs with their associated data and explain which array a reported index refers to.

Review 7 – Binary-Search Variants

First and Last Occurrence

Ordinary binary search may return any occurrence. To locate a repeated block, record a match and keep searching for a better boundary.

For $[2, 4, 4, 4, 7, 9, 9]$, target 4 has first index 1 and last index 3.

Goal	After a match at mid
First occurrence	Save mid; continue left with `high = mid - 1`.
Last occurrence	Save mid; continue right with `low = mid + 1`.

Initialize the saved candidate to -1. If present, the count is $last - first + 1$; otherwise it is zero. Each boundary takes $O(\log n)$ time and $O(1)$ working space, without scanning all duplicate positions.

First Value at Least a Target: Lower Bound

The **lower bound** is the first index whose value is at least the target, or n if no such value exists. It is a position, not necessarily an exact match. For $[12, 20, 20, 35, 50]$, target 20 returns 1, target 26 returns 3, and target 60 returns 5.

```
low = 0
high = n - 1
while low <= high:
    mid = low + (high - low) / 2
    if a[mid] < target:
        low = mid + 1
    else:
        high = mid - 1
return low
```

At termination, all positions before `low` hold smaller values, and all positions from `low` onward hold values at least the target. An empty array returns 0. Check the returned position before reading `a[position]`.

A strict boundary: upper bound

The upper bound is the first index with value *greater than* the target, or n. In the lower-bound pseudocode, replace `a[mid] < target` with `a[mid] <= target`. The previous position, when it exists, is the last value at most the target. Finding a boundary does not physically insert an item.

Review 8 – Fixed-Size Sliding Window

Idea and When to Apply It

A fixed window contains exactly k consecutive values. Examples include the quietest three-hour maintenance period and an average count over the latest five intervals. Neighboring windows overlap; their sums differ by one removal and one addition.

Worked Trace

Array: $[6, -2, 4, 1, -3, 5, 2]$; $k = 3$.

Window	Update	Sum
$[6, -2, 4]$	$6 - 2 + 4$	8
$[-2, 4, 1]$	$8 - 6 + 1$	3
$[4, 1, -3]$	$3 - (-2) - 3$	2
$[1, -3, 5]$	$2 - 4 + 5$	3
$[-3, 5, 2]$	$3 - 1 + 2$	4

Pseudocode for the Minimum Sum

```
require 1 <= k <= n
currentSum = sum of a[0] through a[k - 1]
bestSum = currentSum
bestStart = 0
for right from k to n - 1:
    currentSum = currentSum - a[right - k] + a[right]
    if currentSum < bestSum:
        bestSum = currentSum
        bestStart = right - k + 1
return bestSum and bestStart
```

Correctness and Cost

The current sum equals exactly the current length-k window. The update preserves it, and every valid start is visited. Update only on a strict improvement to preserve the earliest tie. Use `>` instead of `<` for a maximum problem.

There are $n - k + 1$ windows. Independent summation uses $O((n - k + 1)k)$ time; the sliding method uses $O(n)$ time and $O(1)$ working space. The same sums can be used to find a maximum, a minimum, or the number meeting a threshold; change the result update to match the requested output.

Signed values and averages

Fixed-sum updates also work with negative values. Initialize the best result from a real window, not zero. Use a `long` sum and divide by `(double) k` for a mean. Storing all means costs output space for $n - k + 1$ values; streaming them does not.

Review 9 – Variable-Size Sliding Window

Idea and Required Conditions

Keep left and right boundaries plus a running state. Change the length to restore a constraint or improve an objective. The following sum patterns require the stated restrictions on values.

Longest Sum-at-Most Window

For **nonnegative values** and budget $S \geq 0$, add the next right value. While the sum exceeds S, remove from the left. Record the length after the window becomes valid.

Array $[4, 2, 0, 3, 1, 2, 5]$, budget 6:

right	left	Valid window after shrinking	Sum	Best
0	0	$[4]$	4	1
1	0	$[4, 2]$	6	2
2	0	$[4, 2, 0]$	6	3
3	1	$[2, 0, 3]$	5	3
4	1	$[2, 0, 3, 1]$	6	4
5	2	$[0, 3, 1, 2]$	6	4
6	6	$[5]$	5	4

The longest length is 4. When reporting indices, retain the first of the tied windows.

Shortest Sum-at-Least Window

For **positive values** and target $S > 0$, expand right. Whenever the sum is at least S, save the candidate length *before* removing a left value. Keep recording and shrinking while the target still holds. For $[3, 1, 4, 2, 6, 1]$, target 8, the shortest qualifying window is $[2, 6]$, length 2.

Why the Total Work Is Linear

Each value enters once and leaves at most once. Both boundaries advance only forward, so total time is $O(n)$ even with an inner `while`; working space is $O(1)$. Longest-at-most shrinks an *invalid* window to restore validity. Shortest-at-least shrinks a *valid* window to improve its length.

Do not drop the value restriction

With nonnegative values, extending cannot decrease the sum and removing cannot increase it. With $[6, -5]$ and budget 2, immediately discarding 6 loses the later valid length-2 window. These templates are not general signed-sum algorithms.

Review 10 – Unique-Character Windows

Idea and When to Apply It

For a sequence of identifiers, keep the longest consecutive region containing no repeated identifier. A set records the characters in the current window. This condition concerns membership rather than a sum.

Worked Trace Using a Set

Sequence: `ABBCDAEFAC`.

right	New	Window after adjustment	left	Best
0	A	A	0	1
1	B	AB	0	2
2	B	B	2	2
3	C	BC	2	2
4	D	BCD	2	3
5	A	BCDA	2	4
6	E	BCDAE	2	5
7	F	BCDAEF	2	6
8	A	EFA	6	6
9	C	EFAC	6	6

Before adding a character already in the set, repeatedly remove the leftmost character until the old occurrence is gone. Then add the new occurrence. Do not reset the whole window on a repeat.

Optional Alternative: Most Recent Index

A map can store the latest index p of each character. On a repeated character, set $left = \max(left, p+1)$, then update that character's latest index. The maximum prevents backward movement when a previous occurrence is outside the current window.

Correctness and Cost

After adjustment, every character in the window is distinct. Compare its length with the best length so far. With expected constant-time hash operations, total expected time is $O(n)$. For an alphabet of size σ, space is $O(\min(n, \sigma))$; for fixed ASCII this is also $O(1)$ with respect to n. An empty string has length 0.

In Problem 11 each ASCII character is one case-sensitive identifier. This avoids requiring Unicode code-point processing. A substring cannot skip positions; a subsequence can. Sorting the string would change the task.

Problem Set at a Glance

Complete the seven core problems and one problem from each choice group. The tasks progress from tracing and implementing fundamental algorithms to adapting them to practical questions. Difficulty labels are approximate.

No.	Problem	Technique	Status
1	Calibration Offset Ordering	Three sorts	Core
2	Repair Production-Time Order	Insertion vs. selection	2 or 8
3	First Sufficient Battery	Linear search	Core
4	Museum Catalogue Lookup	Binary search	Core
5	Packet-Type Occurrence Range	First/last boundaries	Core
6	First Checkpoint After a Time	Upper-bound adaptation	6 or 11
7	Quietest Machine-Service Slot	Fixed-size window	Core
8	Rolling Queue Delay	Moving means	2 or 8
9	Longest Playlist Within a Time Limit	Longest at-most window	Core
10	Shortest Upload Target	Shortest at-least window	Core
11	Longest Unique Channel Segment	Window with a set	6 or 11
12	Archive Record Locator	Sort records; search	Optional

Counting Operations Consistently

Use these definitions for Problems 1 and 2:

- **Key comparison:** one evaluated comparison of data values. Exclude index checks, loop tests, and output/copying work.
- **Swap:** exchange two different positions using a temporary variable. Skip self-swaps. Report swaps for bubble and selection sort.
- **Array write:** one assignment to an array position. A swap uses two such writes. Count every insertion-sort shift and final key placement, even when final placement rewrites the same value. Do not count temporary-variable assignments.

Count the final false key comparison when it is actually evaluated. In insertion sort, a false index check that prevents a value comparison is not a key comparison. Report shifts separately from writes.

One input, separate copies

Give every algorithm a fresh copy of the same original input. Compare sorting counters without counting the protective copy itself. Include that copy when discussing the memory needed by the complete comparison program.

Problem 1 – Calibration Offset Ordering

Focus: Three elementary sorts; Easy; Core

Scenario and Task

A calibration team records signed integer offsets between -1000 and 1000. Produce an **ascending** list so the offsets can be inspected in numerical order. Repeated offsets are legitimate observations; retain every occurrence. Implement **optimized bubble sort, selection sort, and insertion sort**. For each method display the sorted array, key comparisons, array writes, and swaps or shifts. Use at most 2,000 values and the common counter definitions.

Example

```
Input:  [7, -3, 4, 0, -3, 9]
Sorted: [-3, -3, 0, 4, 7, 9]
```

Print the measured counters in addition to the sorted values.

Required Experiment and Tests

Pattern	Values
Unordered	$[7, -3, 4, 0, -3, 9]$
Already in required order	$[-3, -3, 0, 4, 7, 9]$
Opposite order	$[9, 7, 4, 0, -3, -3]$
Nearly in required order	$[-3, -3, 4, 0, 7, 9]$

Run each method on a fresh copy of every input. Include an empty array, a single value, and an all-equal array. For Problems 1 and 2, sorting is performed on working copies; keep the original for fair comparisons.

Analysis Questions

1. Describe the sorted region maintained by each method for the requested order.
2. Why does a no-swap pass permit early termination? State the optimized bubble sort best and worst time.
3. Which method performs $n(n-1)/2$ key comparisons regardless of input order?
4. Compare insertion shifts with selection swaps, using array writes as the common measure.
5. State working space, including separately the copies used by the comparison driver. Explain which standard methods preserve equal-key order.
6. Why should the elementary sorting experiment stay small even though later search/window problems permit larger arrays?

Problem 2 – Repair Production-Time Order

Focus: Insertion vs. selection sort; Easy–Medium; Applied choice

Scenario and Task

A scheduling log normally arrives in chronological order, but delayed entries may be slightly misplaced. Restore **ascending** order using insertion sort. Here each record is represented only by an integer time code; if payloads are added, move the entire record together. Time codes are nonnegative, may repeat, and $n \leq 5{,}000$.

Example

```
Input:  [810, 820, 850, 830, 840, 860]
Output: [810, 820, 830, 840, 850, 860]
```

Required Comparison

Compare your ascending insertion sort with your ascending selection sort on $[1, 2, 4, 3, 5, 6, 8, 7, 9, 10]$. Repeat on ascending and descending arrangements of the same values. Use fresh copies and record key comparisons, writes, insertion shifts, and selection swaps.

Test an empty array, one time code, and repeated time codes. Reuse methods rather than paste a different implementation into every test. Include all shared source files.

Analysis Questions

1. Which positions move when one delayed entry is inserted into the sorted prefix?
2. Why does insertion sort benefit from this pattern? Compare best and worst time with selection sort.
3. When no shift occurs, does final key placement count as a write under this handout's convention?
4. Why does changing the requested order require changing comparisons, not just relabeling output?
5. Explain stability for two equal time codes carrying different payloads. What could be lost if only the keys moved?

Applied choice

Complete **Problem 2 or Problem 8**. Submit Java source and a report section for your selected problem. Optional extra work does not replace a core problem.

Problem 3 – First Sufficient Battery

Focus: Linear search; Easy; Core

Scenario and Task

A technician inspects batteries in shelf order. Each value is the stored charge in whole units. Given a nonnegative requirement t, find the **first original index with charge at least** t. Return -1 when no battery qualifies. Implement a left-to-right conditional linear search and stop at the first qualifying value. Do not sort the shelf order.

Example

```
Charges: [20, 75, 40, 90, 60]
Requirement: 60
Found: true
First sufficient index: 1
Inspections: 2
```

Required Tests

Use requirements 20, 60, 90, and 100 on the example. Include exact equality, repeated charges, an empty shelf, a single battery, and requirement zero. Charges are nonnegative. An empty shelf returns false, index -1, and zero inspections. Count one evaluated threshold check per inspected value.

Analysis Questions

1. Why does the first successful threshold check establish the earliest qualifying shelf position?
2. Explain why testing equality alone would answer a different question.
3. State best/worst time and auxiliary space. Which cases permit immediate return?
4. Why is ordinary binary search invalid on the unsorted shelf even though the comparison uses an ordered threshold?
5. Would sorting before searching preserve the required original-index answer? Explain the extra record information that would be needed.

Problem 4 – Museum Catalogue Lookup

Focus: Iterative binary search; Easy–Medium; Core

Scenario and Task

A catalogue stores distinct integer identifiers in **ascending order**. Implement iterative binary search using inclusive boundaries. Return the exact matching index or -1; do not return a nearby entry when a match is absent.

For test runs, print or record low, high, mid, and the inspected value at each iteration. Count one midpoint inspection per loop iteration; this is not the same as counting each relational expression in the implementation.

Example and Worked Trace

```
Identifiers: [205, 230, 260, 310, 355, 390, 440]
Target: 390
Returned index: 5
```

low	high	mid	a[mid]	Action
0	6	3	310	Set low to 4.
4	6	5	390	Return 5.

Required Comparison and Tests

Look up the last identifier using both a left-to-right *first-match* linear search and binary search. Record their indices and inspected-value counts. If your Problem 3 contract differs, write a small first-match helper for this comparison.

Test the first identifier, an absent value between entries, values below/above the range, and an empty or single-entry catalogue. Sorted order is a precondition; do not rescan to validate it within every query.

Analysis Questions

1. State where a target can still occur during the loop. How does ascending order justify each discarded region?
2. Why must both updates move past mid? What happens when low equals high?
3. Compare best and worst time, iterative auxiliary space, and worst-case inspections for a million entries.
4. If the source data are unordered, what preparation and memory must be included in a complete repeated-query comparison?
5. Would appending an out-of-order identifier preserve the search contract? Explain.

Problem 5 – Packet-Type Occurrence Range

Focus: First/last binary-search boundaries; Medium; Core

Scenario and Task

A packet report has already grouped type codes by sorting them in nondecreasing order. Find the first index, last index, and total frequency of a requested type. These indices belong to the sorted report, not the original arrival log. Implement **two modified binary searches**. After finding a match, retain it as a candidate and continue toward the requested boundary. Do not linearly scan a duplicate block or replace the searches with a map.

Example

```
Sorted values: [2, 2, 2, 5, 7, 7, 9, 9, 9, 9]
Target: 9
First index: 6
Last index: 9
Count: 4
```

Boundary Cases and Tests

For an absent target report first $= -1$, last $= -1$, count $= 0$. Test one match, duplicates at either end, all-equal data, an absent target, and an empty array. Include a trace of one boundary search. Equal values in the original log may occupy several separated blocks. After sorting, they occupy one group. This changes block layout, even though the total frequency is preserved.

Analysis Questions

1. Why does one ordinary binary-search match not identify the whole occurrence range?
2. Why is searching left/right after saving a match safe?
3. Derive last $-$ first $+1$ and explain the guard needed for absence.
4. Compare the two searches with a full scan when one value repeats many times.
5. For repeated queries, separate sorting cost from per-query cost. What changes if the input is already maintained in sorted order?
6. Which information is lost if original order or original record positions are discarded?

Problem 6 – First Checkpoint After a Time

Focus: Boundary binary search; Medium; Interview choice

Scenario and Task

A monitoring log stores checkpoint times in ascending order. Given time t, return the **first index whose time is strictly greater than** t. A checkpoint at exactly t does not qualify. Return n if no later checkpoint exists and display `NO CHECKPOINT` instead of reading `a[n]`. Do not insert or change any value.

Examples

Use $[12, 20, 20, 35, 50]$.

Target	Result	Meaning
20	3	Skip both entries equal to 20.
26	3	35 is the first later checkpoint.
5	0	The first checkpoint is already later.
60	5	No checkpoint is later; report no result.

Suggested Reasoning and Tests

This position is the upper bound. Adapt the lower-bound pattern: values *at most* t belong on the discarded left side. For an empty array the boundary position is zero and there is no checkpoint to display.

Test exact matches, repeated equal times, gaps, targets outside the range, empty data, and one entry. Trace a case in which the target is absent but a qualifying neighboring time exists.

Analysis Questions

1. Why does this task need an upper bound rather than a lower bound?
2. What must happen when the midpoint equals the target?
3. State what is known about positions before low and beyond high during the search.
4. Why is n a meaningful boundary but an invalid index to dereference? Include the empty case.
5. State lookup time and working space. Explain why no array shifting is needed.

Interview choice

Complete **Problem 6 or Problem 11**. Submit Java source and a report section for your selected problem. Optional extra work does not replace a core problem.

Problem 7 – Quietest Machine-Service Slot

Focus: Fixed-size window; Easy–Medium; Core

Scenario and Task

A workshop records a nonnegative load for each consecutive production slot. Maintenance must occupy exactly k slots. Find the window with the **smallest total load**; when totals tie, choose the earliest start.

Implement **A: independently sum each window** and **B: compute the first sum once and slide**. Return the minimum total as `long` and its zero-based start/end indices. Keep the time order unchanged. Require $1 \leq k \leq n$.

Example

```
Loads: [8, 2, 5, 1, 4, 2, 7]
k: 3
Window sums: [15, 8, 10, 7, 13]
Minimum total: 7
Start index: 3
End index: 5
```

Required Tests and Comparison

Test $k = 1$, $k = n$, all zeros, all equal positive loads, an earliest tie, and a total larger than `int`. On $[2, 1, 2, 1]$ with $k = 2$, all sums equal 3, so choose indices 0–1. Reject $k = 0$, $k > n$, and an empty input with no valid window.

Record all window sums for one normal input from both methods. Include the standard comparison table. No wall-clock benchmark is required.

Analysis Questions

1. How many valid windows exist? Derive the direct work as $(n - k + 1)k$.
2. Why can one subtraction and one addition replace the next full summation?
3. Why must the initial minimum come from a real window rather than zero?
4. Which strict comparison preserves the earliest tied start?
5. State time and working/output space for both methods.
6. Contrast one fixed window length with Lab 1's sum over subarrays of every length. Why does its contribution formula not directly select this minimum?

Problem 8 – Rolling Queue Delay

Focus: Moving averages; Medium; Applied choice

Scenario and Task

A service centre records integer queue-delay readings at equally spaced times. Produce each moving mean and locate the **lowest-average** period. These are illustrative readings, not a causal performance study. Use one fixed-size sliding window with a `long` running sum. For fixed k, compare exact sums to select the best mean and retain the earliest start in a tie. The observations are nonnegative. Require $1 \leq k \leq n$.

Example

```
Observations: [12, 6, 9, 3, 6, 12]
k: 3
Moving means: 9.0000, 6.0000, 6.0000, 7.0000
Lowest mean: 6.0000
Start index: 1
End index: 3
```

Implementation and Tests

Display means to four decimal places, but do not use rounded values for comparisons. Divide by `(double) k`. Either print each mean immediately or return an output array; explain the storage choice.

Test $k = 1$, $k = n$, zero counts, a fractional mean, constant values, a tie, and sums beyond `int`. Reject invalid k. For a constant series, report the first complete period, not the last.

Analysis Questions

1. How many complete means are produced? Which positions are in the first and final windows?
2. Why can exact sums be compared instead of rounded means when all windows have the same length?
3. What changes between minimizing and maximizing the mean?
4. What error results from integer division before conversion?
5. State time, working space, and output storage. Explain why sorting would change the meaning of these periods.

Applied choice

Complete **Problem 2 or Problem 8**. Submit Java source and a report section for your selected problem. Optional extra work does not replace a core problem.

Problem 9 – Longest Playlist Within a Time Limit

Focus: Longest sum-at-most window; Medium; Core

Scenario and Task

A playback queue lists nonnegative durations in a fixed order. A listener chooses a contiguous segment whose total duration is no more than limit S. A zero-duration marker remains part of the sequence. Find the **maximum number of consecutive entries**. Return length and zero-based start/end indices, preserving the earliest start when lengths tie. Use a variable-size sum window; $S \geq 0$. Do not reorder or skip entries.

Example

```
Values: [5, 0, 2, 4, 1, 1, 3]
Budget: 8
Maximum length: 5
Start index: 1
End index: 5
Selected total: 8
```

Suggested Reasoning and Tests

Add the next rightmost value. While the sum exceeds the limit, remove the leftmost value and advance left. Update the best length *after* validity has been restored. A single value greater than the limit may leave an empty window.

For no nonempty result, return `{0,-1,-1}`. Test an empty array, a full-array answer, every positive value above the limit, zeros with limit zero, tied lengths, and a sum beyond `int`. $[0, 0, 5, 0]$ with $S = 0$ returns length 2, indices 0–1. Trace one run requiring several removals.

Analysis Questions

1. State what the current window means after shrinking.
2. Why is the condition sum $> S$, rather than sum $\geq S$?
3. Why do nonnegative values justify permanently discarding a left boundary?
4. Show that each endpoint advances at most n times. State time and auxiliary space.
5. Explain the earliest-tie rule and the no-result marker.
6. Give a signed-value counterexample for this exact method. Identify the discarded start that later becomes useful.

Problem 10 – Shortest Upload Target

Focus: Shortest sum-at-least window; Medium; Core

Scenario and Task

An upload log contains positive data volumes for consecutive completed transfers. Find the shortest consecutive group whose total volume reaches at least target S. Transfers cannot be skipped or split. Use a variable-size sliding window and a `long` running sum, with positive values and $S > 0$. Return minimum length and zero-based start/end indices. If several shortest windows exist, retain the earliest start; return `{0,-1,-1}` when no window reaches the target.

Example

```
Values: [5, 1, 2, 7, 3, 4]
Target: 10
Minimum length: 2
Start index: 3
End index: 4
Selected total: 10
```

Suggested Reasoning

Expand the right boundary. Whenever the sum reaches the target, record the current candidate before removing a leftmost value. Continue saving and shrinking *while the sum is at least the target*. Keep a sentinel for no answer yet, and convert it to length zero only at the end.

Unlike Problem 9, shrinking here improves an already valid window. Updating only after an invalid window has been produced loses the candidate.

Required Tests

Include an empty array, unreachable target, one-entry answer, whole-array-only answer, tied minimum lengths, several removals after one expansion, and totals beyond `int`. Inputs satisfy positivity; parsing invalid data is outside the task. Trace left, right, sum, and the saved result on one nontrivial case.

Analysis Questions

1. Why must a candidate be recorded before the next removal?
2. Why can a single if statement be insufficient in place of the shrinking while loop?
3. Explain how positivity supports the reasoning; do not claim the method handles arbitrary signed data.
4. Compare the objective, condition, and update moment with Problem 9.
5. State time and auxiliary space and justify forward-only endpoint movement.
6. How do you distinguish no result from a length-one answer at index zero?

Problem 11 – Longest Unique Channel Segment

Focus: Character window with hashing; Medium; Interview choice

Scenario and Task

A receiver records channel labels as a string, one ASCII character per label. Return the length of the longest contiguous segment containing no repeated label. Comparison is case-sensitive. Use a sliding window with `HashSet<Character>`; do not generate every substring. An empty string returns zero. Reporting the substring itself is optional.

Examples

Sequence	Length	One longest segment
`PQRPSTP`	5	`QRPST`
`zzzz`	1	`z`
`cabdca`	4	`cabd`
`aAbBc`	5	`aAbBc`
empty string	0	*none*

Required Tests and Optional Map Method

Test empty, one-character, all-equal, all-distinct, mixed-case, and repeated labels after earlier removals. Trace at least one window. Before adding a repeated character, remove leftmost characters until its earlier occurrence is gone.

Optional: use a map of most recent indices. Update left to `max(left, previous + 1)`. Test `abba`; its answer is 2. An occurrence outside the current window must not move left backward.

Analysis Questions

1. What does the set represent, and why is the window unique after adjustment?
2. Why does discarding the entire window at every repeat lose possible answers?
3. State expected time and space, including the expected constant-time hashing assumption.
4. For a fixed ASCII alphabet, explain why storage is bounded even if the string is long.
5. Explain substring versus subsequence using one example. Does sorting preserve this task?
6. If you attempted the map method, explain the forward-only left boundary and compare its stored state with the set.

Interview choice

Complete **Problem 6 or Problem 11**. Submit Java source and a report section for your selected problem. Optional extra work does not replace a core problem.

Problem 12 – Archive Record Locator

Focus: Records; sorting and repeated searching; Medium–Hard; Optional

Scenario and Task

Build a small lookup tool for unordered records. Each record has a unique integer ID and an associated integer `shelfNumber`. Sort a protective copy of the records once by ID, then answer several exact-ID queries. Use at most 2,000 records for the insertion-sort experiment; discuss larger workloads without running a large quadratic sort.

Required Program

1. Represent a complete record with its ID and payload, for example a `ArchiveRecord` object.
2. Implement ascending insertion sort comparing IDs and moving complete records, not just their ID fields.
3. Use binary search on the sorted records. Display the matching ID and payload, or `NOT FOUND`.
4. Implement a linear-search baseline on the original records and check that every query returns the same payload or absence.

Example

```
Original (ID, payload): [(708, 12), (502, 3), (901, 8), (604, 6)]
Sorted (ID, payload):   [(502, 3), (604, 6), (708, 12), (901, 8)]
Query 604: 604 6
Query 605: NOT FOUND
```

Tests and Analysis

Test present and absent IDs, empty and one-record collections, and ascending/reverse input order. Preserve every ID–payload relationship and keep the original records unchanged.

1. Why would sorting only the IDs corrupt lookup results?
2. For n records and q queries, compare repeated scans with insertion sort once followed by binary searches. Include preparation and protective-copy memory.
3. Why is the insertion-based worst case $O(n^2 + q \log n)$, not automatically $O(n \log n + q \log n)$?
4. How does one query differ from many queries on the same fixed data? What becomes invalid if the records change?

Interview Corner – Reasoning and Debugging

Complete **Interview Corner A–D as written answers in your PDF lab report**. Include the requested reasoning, traces, and short pseudocode or corrected code snippets where appropriate. **Separate runnable Java programs are not required for A–D.**

This written section is separate from the **Interview Choice** in the programming problem set. For your selected **Problem 6 or Problem 11**, submit both the Java implementation and the corresponding report section.

Interview A – First Unique Value or First Unique Position?

Lab 1 asked for the first original position whose value occurs once. Consider sorting complete `(value, originalIndex)` pairs and identifying groups of size one. Use original values $[9, 4, 9, 7, 2]$ for the explanation.

1. Why is the numerically smallest unique value not necessarily the earliest unique original position?
2. How could the original indices of singleton groups recover the requested result?
3. Compare insertion-sort preparation plus grouping with the earlier frequency-map and ordered-scan method. Include storage.
4. Explain why sorting changes equal-value blocks but preserves total frequencies. No extra program is required.

Interview B – A Plausible Midpoint Is Not Enough

Someone applies ordinary ascending binary search for 10 to the unordered array $[8, 10, 3, 6, 9]$.

1. Trace the first midpoint and identify the actual target position that is discarded.
2. State the missing precondition. Why do successful tests for a few other values not establish correctness?
3. If a sorted copy is used instead, what must be retained to return an original-position answer?

Interview C – Which Value Leaves the Window?

The first length-k sum has been calculated correctly. Examine:

```
for (int right = k; right < a.length; right++) {
    sum = sum - a[right - k + 1] + a[right];
    // record the current window sum
}
```

Trace $[5, 1, 4, 2]$ with $k = 2$. Compare the first updated result with the sum of indices 1–2. Correct the outgoing index and explain how the corrected update also works when $k = 1$. A short corrected snippet is sufficient.

Interview Corner – Recognizing the Technique

Interview D – Choose the Property, Not the Label

For each situation identify the technique and the property that makes it suitable. For a window, state fixed or variable size and the maintained state. These are written answers, not additional programs.

1. Find the smallest total across three consecutive machine slots.
2. Find the first original index holding a value that appears once overall.
3. Find a longest channel segment without repeated labels.
4. Find a catalogue ID in an ascending array.
5. Find the shortest positive-volume group reaching an upload target.
6. Arrange a small list of signed calibration offsets.
7. Answer arbitrary suffix-maximum queries. Explain why subtracting two maxima does not mimic a range-sum formula.

Technique-Recognition Exercise

For each situation write technique, reason, and expected complexity. Count preparation separately when needed. No additional implementation is required.

Case	Situation
A	Find the first sufficiently charged battery in the original shelf order.
B	An exact lookup in a sorted fixed museum catalogue.
C	All means over consecutive six-reading queue-delay periods.
D	A small ordered production list with two nearby misplaced times.
E	The first position strictly after a threshold in a sorted log.
F	A longest nonnegative-duration playlist segment within a limit.
G	Many suffix-maximum queries on the same array, as in Lab 1.

Choose the right stored information

A suffix maximum, a frequency map, and a running window sum encode different information. Storing a summary once does not make every later question constant-time. Identify exactly which result the summary supports.

Comparison, Reflection, and Common Mistakes

Algorithm Comparison Exercise

Complete the table in your report. State best/worst cases where different, and make value and input-order assumptions explicit. For variable windows, use the sum-based versions from Problems 9 and 10.

Technique	Sorted input required?	Time / space	Main condition
Linear search			
Binary search			
Optimized bubble sort			
Selection sort			
Insertion sort			
Fixed-size window			
Variable-size window			

Reflection Questions

1. Why can linear search be sensible for one query even though binary search has a smaller per-query worst case?
2. What does sorting preserve, and what original-position information may it destroy? Include preparation when discussing its benefits.
3. What arithmetic is shared by adjacent fixed windows? Explain how the same total supports a different reporting objective.
4. Compare the longest-at-most and shortest-at-least shrinking conditions and when each best result is updated.
5. Which technique was hardest to understand? Describe one trace or failing test that clarified it.

Common Mistakes to Check

- Reusing an already sorted output as the next algorithm's input; losing attached payloads; inconsistent counters.
- Binary search without sorted order; failing to move past the midpoint; returning one duplicate as the whole range; reading a lower-bound position equal to n.
- Sorting before window analysis; initializing a minimum or maximum to zero without justification; using integer division for averages.
- Shrinking only once; applying these variable sum windows to signed values; moving left backward in a map-based character window.

Report Requirements

Submit one PDF report showing your full name, student ID, **Lab 2 – Version 4**, completed problem numbers, and the two selected choices. Do not repeat the complete statements or fill the report with source listings.

For Every Completed Programming Problem

1. Explain the algorithm and boundary/missing-result handling.
2. Give a short correctness argument using the sorted region, search interval, or window state.
3. State time and auxiliary space, distinguishing relevant cases, preprocessing, protective copies, and returned output storage.
4. Answer the problem's analysis questions. Discuss optional methods only when attempted.
5. Include the specifically requested traces, counters, and comparisons.
6. Record expected and actual outputs for normal, boundary, and tricky inputs, covering the named cases as well as the minimum of three tests.

Test Record Template

Case	Input	Expected	Actual
Normal			
Boundary			
Tricky			

Comparison Template

Use this for Problems 1, 4, and 7, plus Problem 2 when selected. Add a third algorithm column for Problem 1. Include comparisons for optional methods only when implemented.

Criterion	Approach A	Approach B
Main idea		
Time complexity		
Auxiliary / output space		
Counters, if required		
Advantages and limitations		
Suitable input / workload		

Include **Interview Corner A–D and other written work in this report; no extra programs are required**. The separate 15-minute bonus quiz requires no report section and does not replace lab work.

Submission Requirements and Final Checklist

Submission Package

Submit `StudentID_FullName_Lab02_V4.zip` containing Java source, one PDF report, and `README.txt` with the Java version and exact compile/run commands. Include only completed choice/optional folders. Shared helpers must be present; compiled `.class` files and IDE build folders are unnecessary.

```
StudentID_FullName_Lab02_V4.zip
  src/
    Problem01_CalibrationOffsets/
    Problem03_BatteryScan/
    Problem04_MuseumCatalogue/
    Problem05_PacketRange/
    Problem07_ServiceWindow/
    Problem09_PlaylistBudget/
    Problem10_UploadTarget/
    Problem02_ProductionTimes/      (choose 02 or 08)
    Problem08_QueueDelay/      (choose 02 or 08)
    Problem06_NextCheckpoint/  (choose 06 or 11)
    Problem11_ChannelSegment/    (choose 06 or 11)
    Problem12_ArchiveRecords/    (optional)
    common/                    (if used)
  report/
    Lab02_V4_Report.pdf
  README.txt
```

Final Checklist

- Seven core problems and one from each choice group are complete.
- Java files compile and run with the README instructions; required methods are implemented manually.
- Explanations cover idea, correctness, time, memory, and the requested questions.
- Required traces, counters, comparisons, and recorded test outputs are present.
- Search receives correctly ordered data; windows preserve original adjacency and respect value/size restrictions.
- Interview Corner A–D is report-only; the ZIP includes the PDF and source.

Final objective

Identify the property that avoids repeated work. Explain why that property holds for the stated input before choosing an algorithm.